Byzantine Fault Tolerance

CSIS 604 - Distributed Systems

Dan Johnson

- The Byzantine Generals Problem and Byzantine Fault Tolerance
  - Introduction
  - Analysis of $OM(2)$ for $n = 7, m = 2$
    - Case 1: The Commander Is Loyal
      - Top-Level Execution of $OM(2)$
      - Recursive Level: $OM(1)$
      - Base Case Level: $OM(0)$ and Upward Voting
      - Top-Level Vector Aggregation and Decision ($OM(2)$)
    - Case 2: The Commander Is a Traitor
      - Top-Level Execution of $OM(2)$
      - Recursive Level: $OM(1)$
      - Base Case Level: $OM(0)$ and Upward Voting
      - Top-Level Vector Aggregation and Decision ($OM(2)$)
  - Conclusion
- References

The Byzantine Generals Problem and Byzantine Fault Tolerance

Introduction

The Byzantine Generals Problem is a classic thought experiment involving the fault tolerance of distributed systems. The fundamental actors in the thought experiment are *generals* commanding armies laying siege to an enemy city. These generals must come to consensus through some means to decide whether to attack or retreat which is subject to the passing of messages. Let G be the set of generals, then for $n = |G|$ suppose that there exists some $T \subseteq G$ of *traitors*, with $m = |T|$. According to the seminal work on the problem, an algorithm that robustly provides a solution to this problem (known as Byzantine Fault Tolerance or BFT) must involve $n \geq 3m + 1$ generals.[1] Lamport et al. constructed such an algorithm, known as *Oral Message* for up to m traitors, denoted $OM(m)$. While previous demonstrations have shown that $OM(1)$ for $n = 4$ provides BFT, subsequent demonstrations have been left to the reader. Thus, a demonstration that $OM(2)$ provides BFT for $n = 7$ is provided below.

Analysis of $OM(2)$ for $n = 7, m = 2$

To illustrate the mechanics of $OM(2)$ in a system where $|G| = 7$ and $|T| = 2$, two distinct scenarios must be considered:

1. The commander $C \in G$ is loyal ($C \notin T$)
2. The commander is a traitor ($C \in T$)

The algorithm must satisfy two key *interactive consistency* criteria in order to provide BFT[2].

1. $IC1$: *All loyal lieutenants obey the same order*[3].
2. $IC2$: *If the commanding general is loyal, then every loyal lieutenant obeys the order he sends*[4].

Notably, $IC2$ is only applicable *if the commanding general is loyal*, and also $IC1$ naturally follows from $IC2$[5].

[1] Leslie Lamport et al., "The Byzantine Generals Problem," *ACM Transactions on Programming Languages and Systems* (New York, NY, USA) 4, no. 3 (1982): 382–401, https://doi.org/10.1145/357172.357178.
[2] Lamport et al., "The Byzantine Generals Problem."
[3] Lamport et al., "The Byzantine Generals Problem."
[4] Lamport et al., "The Byzantine Generals Problem."
[5] Lamport et al., "The Byzantine Generals Problem."

Case 1: The Commander Is Loyal

Suppose that the commanding general C is loyal, and broadcasts the initial order $v \in \{\text{ATTACK}, \text{RETREAT}\}$ to each of the remaining lieutenants in $G - \{C\}$. There are 6 lieutenants denoted L_i for $1 \leq i \leq 6$ and precisely two of them are traitors. Fix $k, h \in [6]$ with $k \neq h$ such that $T = \{L_k, L_h\}$.

Top-Level Execution of $OM(2)$

1. **Initial Broadcast:** The commander C sends the value v to each lieutenant $L_i \in \{L_1, \ldots, L_6\}$. Because C is loyal, every lieutenant receives precisely the order $v_i = v$.

2. **Delegation to** $OM(1)$**:** Each lieutenant L_i acts as a *sub-commander* to relay their received value v_i to the remaining 5 lieutenants by initiating a sub-instance of $OM(1)$.

Recursive Level: $OM(1)$

1. **Sub-delegation Phase:** Each sub-commander L_i sends their value v_i to the other 5 lieutenants.

   - If L_i is loyal ($i \notin \{k, h\}$), L_i faithfully transmits $v_i = v$ to all peers.
   - If L_i is traitorous ($i \in \{k, h\}$), L_i may send arbitrary conflicting messages or fail to send a message (defaulting to RETREAT).

2. **Delegation to Base Case** $OM(0)$**:** Each receiving lieutenant L_j ($j \neq i$) acts as a sub-sub-commander, invoking $OM(0)$ to relay what they received from L_i to the remaining 4 lieutenants.

Base Case Level: $OM(0)$ **and Upward Voting**

1. **Execution of** $OM(0)$**:** Direct delivery occurs. Each node receives messages from its peers and records the values or defaults to RETREAT if no message is received.

2. **Voting at Rung** $OM(1)$**:** For a given sub-commander L_i, each loyal lieutenant L_j aggregates the values received via $OM(0)$ across the 5 peers and applies the local majority function:

$$v_{i,j} = \text{majority}(v_{i,1}^{(0)}, \ldots, v_{i,5}^{(0)})$$

For any loyal sub-commander note that there are only 2 traitors, with $5-2=3$, thus majority will *always* produce v for $v_{i,j}$.

Top-Level Vector Aggregation and Decision ($OM(2)$) At the top level $OM(2)$, each loyal lieutenant L_j constructs their final vector of length 6 using the resolved $OM(1)$ majority outputs:

$$\vec{v}_j = [v_{1,j}, \dots, v_{6,j}]$$

Since C is loyal, all 4 loyal lieutenants contribute v to their respective entries in $\vec{v}_j$. Therefore, at least 4 out of the 6 elements in every loyal lieutenant's vector are equal to v. Evaluating the top-level majority function yields:

$$\text{majority}(\vec{v}_j) = v \ \forall j \notin \{k, h\}$$

Thus, all loyal lieutenants reach the exact same decision sent by the commander, satisfying $IC2$ (and consequently $IC1$).

Case 2: The Commander Is a Traitor

Suppose that the commanding general C is a traitor ($C \in T$). Because the total number of traitors is capped at $m = 2$, there can be at most **one additional traitor** among the six lieutenants [6]. Fix $k \in [6]$ such that $T = \{C, L_k\}$, meaning L_1 through L_6 (excluding L_k) are loyal lieutenants.

Top-Level Execution of $OM(2)$

1. **Initial Broadcast:** The traitorous commander C sends arbitrary, potentially conflicting values v_i to each lieutenant $L_i \in \{L_1, \dots, L_6\}$ [7].

2. **Delegation to $OM(1)$:** Each lieutenant L_i acts as a sub-commander to relay their received value v_i to the remaining 5 lieutenants by initiating a sub-instance of $OM(1)$ [8].

[6] Lamport et al., "The Byzantine Generals Problem."
[7] Lamport et al., "The Byzantine Generals Problem."
[8] Lamport et al., "The Byzantine Generals Problem."

Recursive Level: $OM(1)$

1. **Sub-delegation Phase:** Each sub-commander L_i sends their value v_i to the other 5 lieutenants.

   - If L_i is **loyal** ($i \neq k$), L_i faithfully transmits v_i to all peers.
   - If L_i is **traitorous** ($i = k$), L_k may send conflicting messages to different peers or fail to transmit.

2. **Delegation to Base Case** $OM(0)$**:** Each receiving lieutenant L_j ($j \neq i$) acts as a sub-sub-commander and invokes $OM(0)$ to relay what they received from L_i to the remaining 4 lieutenants.

Base Case Level: $OM(0)$ **and Upward Voting**

1. **Execution of** $OM(0)$**:** Direct delivery occurs. Nodes record incoming scalar values or fall back to RETREAT if unreceived.

2. **Voting at Rung** $OM(1)$**:** For a given sub-commander L_i, each loyal lieutenant L_j ($j \neq k$) aggregates the values received via $OM(0)$ across their peers and applies the local majority function:

$$v_{i,j} = \text{majority}(v_{i,1}^{(0)}, \dots, v_{i,5}^{(0)})$$

   - For any **loyal** sub-commander L_i ($i \neq k$), all lieutenants in that sub-instance (except L_k) are loyal. Since $5 - 1 = 4$ nodes faithfully relay v_i, majority yields $v_{i,j} = v_i$ for every loyal L_j.
   - For the **traitorous** sub-commander L_k, although L_k attempts to send conflicting values, the recursive cross-examination via $OM(0)$ ensures consistency. By Lemma 1, because at least 4 of the 5 lieutenants in that sub-instance are loyal, every loyal lieutenant computes the **exact same majority value** $v_{k,j} = v_k^*$[9].

Top-Level Vector Aggregation and Decision ($OM(2)$**)**

At the top level $OM(2)$, each loyal lieutenant L_j ($j \neq k$) constructs their final vector of length 6 using the resolved $OM(1)$ majority outputs:

$$\vec{v}_j = [v_{1,j}, v_{2,j}, \dots, v_{6,j}]$$

[9]Lamport et al., "The Byzantine Generals Problem."

Because $v_{i,j} = v_i$ for all loyal L_i, and $v_{k,j} = v_k^*$ for the traitorous L_k, **every loyal lieutenant constructs the exact same vector**:

$$\vec{v}_1 = \vec{v}_2 = \cdots = \vec{v}_5 = [v_1, v_2, \ldots, v_k^*, \ldots, v_6]$$

Evaluating the top-level majority function over these identical vectors yields:

$$\text{majority}(\vec{v}_j) = \text{majority}(\vec{v}_i) \quad \forall i, j \notin T$$

Thus, despite the commander's treason and initial message diversification, all loyal lieutenants decide upon the exact same final plan of action, satisfying Interactive Consistency condition **IC1**.

Conclusion

Due to the recursive nature of the $OM(m)$ algorithm, BFT can be guaranteed for up to m traitors, but this is not without costs. In fact, to sustain this level of fault tolerance, $(n-1)(n-2)\cdots(n-m-1)$ messages may be involved[10]. This scaling limitation may prove to be too much for many applications, "However, when extremely high reliability is required, such assumptions cannot be made, and the full expense of a Byzantine Generals solution is required."[11]

[10] Lamport et al., "The Byzantine Generals Problem."
[11] Lamport et al., "The Byzantine Generals Problem."

References

Lamport, Leslie, Robert Shostak, and Marshall Pease. "The Byzantine Generals Problem." *ACM Transactions on Programming Languages and Systems* (New York, NY, USA) 4, no. 3 (1982): 382–401. https://doi.org/10.1145/357172.357178.